

How Etzan Works

Scoring, machine learning in the browser, and the trend forecast

Etzan (اتزان) · Tuwaiq Academy data-science project · Technical reference

Etzan measures how screen use affects focus and sleep. It has three tools, and they do **not** work the same way internally — a distinction that explains most of the confusion about what any given number means. This document covers all three: what runs the models, how raw model output becomes a level, and what the forecast line is.

- 1

The one scale everything speaks
- 2

Two engines, not one
- 3

How ONNX works here
- 4

The six models
- 5

Feature encoding: the hard part
- 6

Model output → level
- 7

The tracker's scoring (no ML)
- 8

The forecast line
- 9

Limits and honest caveats

1. The one scale everything speaks

Internally Etzan computes in three different units: classifier probabilities (0–1), a sleep-quality regression (1–10, higher is better), and a tracker balance score (0–100, higher is better). Showing all three would force the reader to reconcile three scales pointing in two directions.

So every number is converted to **one canonical scale before display**: five named levels where a *higher level means more concern*. This lives in `lib/scoring.ts`, the single source of truth.

LEVEL	NAME (EN / AR)	COLOUR	MEANING
1	Excellent · ممتاز	sage	Nothing that calls for attention
2	Good · جيد	sage	Healthy, minor room to improve
3	Moderate · متوسط	butter	Middle of the scale
4	Needs attention · يحتاج انتباه	blush	A pattern worth changing
5	Concerning · خطر	blush	Strong signal across several inputs

Charts plot a *continuous* position on this scale (e.g. 2.47) so a trend line can move between levels, and the `reversed` y-axis puts level 1 at the top — so on every chart, **up means better**.

2. Two engines, not one

This is the single most useful thing to know about the codebase.

TOOL	ROUTE	ENGINE	WHAT PRODUCES THE NUMBER
Brain-rot check	<code>/brain</code>	ONNX models	2 trained classifiers
Sleep check	<code>/sleep</code>	ONNX models	4 trained models (2 classifiers, 2 regressors)
Daily tracker	<code>/dashboard</code>	Plain arithmetic	Two hand-written weighted formulas

KEY POINT

The tracker **never calls a model**. Its balance score, its trend chart and its green forecast line are all ordinary arithmetic over what you typed. The six ML models power only the two checks. Conflating the two is why the forecast can look more authoritative than it is.

3. How ONNX works here

Why ONNX at all

The models are trained in Python with scikit-learn, but Etzan has **no server** — it is a static site on GitHub Pages. ONNX (Open Neural Network Exchange) is the bridge: a portable file format for a trained model plus its inference graph. Train in Python, export to `.onnx`, execute in the browser with `onnxruntime-web`.

The consequence is the privacy property the whole product is built on: **your answers never leave the device**, because there is nowhere to send them.

Build time — `etzan/export/export_onnx.py`

CSV datasets

```
→ pandas feature engineering + get_dummies
→ sklearn model .fit()          (RANDOM_STATE = 42, 80/20 stratified split)
→ skl2onnx.convert_sklearn()    (FloatTensorType [None, n_features])
→ public/models/<name>.onnx      + <name>.meta.json
```

Two details in the conversion matter downstream:

- `FloatTensorType([None, n_features])` — the first dimension is dynamic, which is what lets the bedtime sweep push 21 rows through in a single call.
- `options={id(clf): {"zipmap": False}}` — without this, a scikit-learn classifier exports its probabilities as a ZipMap (a list of dictionaries), which is awkward to read from JavaScript. Disabling it makes the model emit two plain tensors: `[label, probabilities]`.

Every estimator is seeded with `RANDOM_STATE = 42`, so re-running the export reproduces the same files and the same scores — the reported numbers are exact, not approximate.

Run time — `src/lib/onnx.ts`

The runtime is configured for a constrained, offline, no-server environment:

```
import * as ort from "onnxruntime-web/wasm";
import wasmSimdThreaded from "onnxruntime-web/ort-wasm-simd-threaded.wasm?url";

ort.env.wasm.wasmPaths = { wasm: wasmSimdThreaded }; // bundled, never a CDN
ort.env.wasm.numThreads = 1; // avoids COOP/COEP headers Pages cannot set
ort.env.wasm.proxy = false;
```

- **WASM only**, no WebGPU/JSEP. The `.wasm` binary is bundled by Vite and served from the same origin, so the app works with no external requests at all.
- **Single-threaded on purpose**. Multi-threaded WASM needs `SharedArrayBuffer`, which requires COOP/COEP response headers. GitHub Pages cannot set headers, so threading is disabled rather than silently broken.
- **Lazy and cached**. `loadModel()` keeps a `Map<string, Promise<LoadedModel>>`. Caching the *promise* (not the resolved value) means two concurrent callers share one download instead of racing.

Inference itself is a single tensor in, tensors out:

```
const input = new ort.Tensor("float32", Float32Array.from(vector), [1, vector.length]);
const outputs = await session.run({ input });

// classifier: zipmap=False -> outputs[1] holds the probability row
const probaKey = session.outputNames[1] ?? session.outputNames[0];
return Array.from(outputs[probaKey].data as Float32Array);
```

Payload cost

The six models total about **7.0 MB**, and the WASM runtime is another **13.5 MB**. That is why routes are code-split and models load lazily: someone landing on the home page downloads neither. Only opening `/brain` or `/sleep` pulls the runtime and the specific models that route needs.

4. The six models

All figures are measured on a held-out test set (20% of the data) by the export script. The "notebook" column is the original baseline from the analysis notebook, which used a different pipeline — the shipped number is the one that matters.

MODEL	ALGORITHM	TYPE	METRIC	NOTEBOOK	SIZE
Brain rot	Logistic Regression	classifier	AUC 0.856	—	0.9 KB
Psychological impact	Gradient Boosting	classifier	AUC 0.811	0.811	157 KB
Sleep quality	Random Forest	regressor	R ² 0.819	0.828	2.13 MB
Sleep disorder	Random Forest	classifier (4-class)	acc 0.865	0.950	2.59 MB
Waking rested	Logistic Regression	classifier	AUC 0.809	—	1.7 KB
Suggested bedtime	Random Forest	regressor	R ² 0.756	0.759	2.11 MB

A DELIBERATE TRADE-OFF

The sleep-disorder model reached **0.95** accuracy with `HistGradientBoostingClassifier`, but that estimator does not convert to ONNX. A Random Forest at **0.865** ships instead. Nine points of accuracy were traded for the ability to run entirely in the browser with no server — which is the product's core promise. The linear models are kilobytes; the forests are megabytes because every tree is serialised.

The bedtime sweep

The "ideal bedtime" is not a model output. The bedtime recommender is a plain regressor that predicts sleep quality for one candidate schedule, so Etzan asks it 21 times — every duration from 5 h to 10 h in 0.25 h steps — in one batched call, then picks the argmax:

```
const { durations, rows } = bedtimeCandidates(raw, bMeta); // 21 rows
const sweepQuality = await predictValues("bedtime_recommender", rows);
// tensor shape [21, n_features] -> one session.run(), not 21

let bestIdx = 0;
for (let i = 1; i < sweepQuality.length; i++) {
  if (sweepQuality[i] > sweepQuality[bestIdx]) bestIdx = i;
}
const bedtime = wakeUpTime - durations[bestIdx]; // then subtract from wake-up
```

That sweep is exactly what the curve on the sleep result page draws. The green dot is the argmax.

5. Feature encoding: the hard part

An ONNX model is just a graph of matrix operations. It has no idea what a column name is — it takes a flat `float32` vector in one exact order. Everything pandas did at training time must therefore be reproduced *by hand in TypeScript*, identically, or the numbers are silently wrong rather than obviously broken.

That contract is what each `<name>.meta.json` carries. Etzan uses two encodings:

Direct (brain-rot models)

Every input is already numeric or a simple 0/1 flag. The metadata ships `feature_order`, and encoding is a lookup in that order:

```
export function orderDirect(record: NumRecord, featureOrder: string[]): number[] {
  return featureOrder.map((k) => record[k] ?? 0);
}
```

get_dummies (sleep models)

The sleep dataset has real categorical columns — occupation, country, chronotype, season. Training used `pandas.get_dummies(drop_first=True)`, so the TypeScript side must rebuild the same one-hot columns in the same order. The metadata carries `dummy_columns` (the exact trained column list) and `categorical_vocab` (the levels each categorical column had):

```
return dummyColumns.map((name) => {
  const hit = lookup.get(name); // e.g. "season_Winter"
  if (hit) return String(raw[hit.col]) === hit.value ? 1 : 0;
  return Number(raw[name] ?? 0); // plain numeric column
});
```

Two supporting mechanisms fill the gaps:

- **Engineered features are recomputed client-side** — `work_stress_load` (work hours × stress), `stimulant_load`, `disruption_score`, `restorative_sleep_percentage` (REM + deep) and others are derived in `features.ts` using the same training-set maxima as constants.
- **Optional inputs fall back to training medians.** Wearable fields (REM %, deep sleep %, latency, wake episodes, resting HR) are optional in the UI. When left blank they are filled from `sleep_medians.json`, exported alongside the models — the same "profile fill" the notebook used, so a blank field means "typical", not zero.

6. Model output → level

Three conversion functions, all in `lib/scoring.ts`.

Probability → level

```
export function levelFromProbability(probability: number): Level {
  const displayed = Math.round(probability * 100) / 100; // round FIRST
  const thresholds = [0.2, 0.4, 0.6, 0.8];
  return toLevel(1 + thresholds.filter((t) => displayed >= t).length);
}
```

WHY THE ROUNDING COMES FIRST

Banding the raw value let a probability of 0.398 fall in the 0.2-0.4 band (level 2, green, "balanced") while the very same value rendered on screen as **"40%"** — which reads as the next band up. The screen then showed a green verdict beside a number that contradicted it. Rounding to display precision before banding makes the verdict and the printed number agree by construction.

Sleep quality (1-10) → level

```
levelPositionFromQuality10 = clamp(6.5 - (quality + 1) / 2, 1, 5)
```

The scale inverts, because 10 is the *best* quality but level 1 is the *best* level. Kept continuous so the duration curve can be drawn smoothly; the card's discrete level is derived from the same number, so a card and the curve beside it can never disagree.

Balance (0-100) → level

```
levelFromBalance100 = 1 + 4 × (1 - clamp(balance, 0, 100) / 100)
```

A linear inversion: balance 100 → level 1, balance 0 → level 5.

7. The tracker's scoring (no ML)

The daily tracker asks eight questions and turns them into two 0-100 sub-scores using hand-written weighted formulas. Every input is first normalised to 0-1, then weighted:

```
brain = 0.30·screen + 0.25·social + 0.30·compulsive + 0.15·stress  
sleep = 0.40·hours + 0.22·stress + 0.13·caffeine + 0.10·exercise + 0.15·rested  
  
balance = (brain + sleep) / 2
```

The normalisations encode the assumptions, and they are worth stating plainly:

INPUT	NORMALISATION	ASSUMPTION BEING MADE
Sleep hours	$1 - h - 8 / 4$	8 h is ideal; falls off either side over 4 h
Screen hours	$1 - h / 10$	10 h+ scores zero
Social hours	$1 - h / 6$	6 h+ scores zero
Caffeine	$1 - \text{mg} / 400$	400 mg is the practical ceiling
Stress	$1 - (s - 1) / 9$	linear over the 1-10 input
Compulsive use	$1 - (c - 1) / 4$	linear over the 1-5 input
Exercise / rested	0 or 1	flat bonus

These weights are a considered judgement, not a fitted result. Nothing was trained here. The value of the tracker is *relative* — watching your own number move — not absolute.

8. The forecast line

The dashed green line on the tracker is an **extrapolation, not a prediction**, and no model is involved. It answers exactly one narrow question:

"If the average direction of the last 7 days simply continued, where would you be?"

The four steps, with a worked example

Take a real week that dips mid-week and recovers:

DATE	19 JUL	20	21	22	23	24	25
brain	70.9	56.7	33.6	14.1	37.2	56.7	74.5
sleep	87.5	69.6	36.6	25.7	52.9	84.6	93.2
balance	79.2	63.2	35.1	19.9	45.1	70.7	83.9

Step 1 — score each day. The formulas in section 7 give the balance row above.

Step 2 — fit one straight line. Ordinary least squares through those 7 points yields a single number, the slope: `+1.396` balance points per day. That is the average direction of the week, nothing more.

Step 3 — extend from today. The projection is anchored at the *last observed value*, not at the fitted line's own height:

```
projection(k) = lastValue + slope × k          for k = 1 .. 7
               = 83.9 + 1.396 × k
               → 85.3, 86.7, 88.1, 89.5, 90.9, 92.3, 93.7  (clamped to 0..100)
```

WHY ANCHORING MATTERS

Evaluating the regression at the projected indices instead — the obvious implementation — starts the dashed line wherever the fitted line happens to sit on the final day. On this V-shaped week that is about **60.9**, far below the **83.9** the solid line ends on. The two series then met at a visible step, and the drawn line dropped while the badge said "improving". Anchoring removes the discontinuity and makes the picture agree with the text.

Step 4 — convert and label. `93.7` → level `1 + 4×(1 - 0.937) = 1.25` → rounds to level 1, *Excellent*. The badge compares the drawn endpoint to today: `93.7 - 83.9 = +9.8`, past the ± 3.5 "steady" band, so it reads *improving*. Deriving the badge from the endpoint that is actually drawn — including any clamping — means the badge cannot contradict the line above it.

Dates and the time axis

Both tracker charts plot on a real time axis (epoch milliseconds), not a categorical one, so equal pixel distance is equal elapsed time. If you skip three days, the chart shows a three-day gap rather than pretending the points are adjacent. All date arithmetic is done in UTC: stepping a day through a *local-midnight* `Date` and serialising with `toISOString()` silently loses a day everywhere east of UTC.

9. Limits and honest caveats

- **The forecast is a straight line by construction.** It cannot bend, cannot capture a weekly rhythm, and knows nothing about next week's exam.
- **A straight line through a V is dominated by its endpoints.** The worked example reads "+1.4/day improving" mainly because the 25th ended above the 19th. With only 7 points,

one bad day swings the slope hard.

- **The tracker is circular by design.** It scores what you typed, so logging less screen time raises the line. It measures reported habits, not an outcome.
- **The datasets are cross-sectional and largely self-reported.** They show correlations, not causes. A high score does not mean screens *caused* the state.
- **Model accuracy is not certainty.** An AUC of 0.856 is a useful signal across a population, not a verdict about one person on one day.
- **Etzan is an educational project, not a medical device.** It must not be used to diagnose a condition or to change a treatment.

PRIVACY, IN ONE LINE

There is no server, no database and no account. Models execute inside your browser and tracker entries are saved to `localStorage` on your device alone. No analytics, no cookies. Clearing browser data deletes your entries permanently — take a backup first.